

Reverse Engineering SaccadeJEPA: From Joint-Embedding Predictive Architectures to Gaze Dynamics Simulation

Emad Bahreinipour

May 2026

Contents

1	General Description of JEPA	2
1.1	The Basic Idea	2
1.2	Core Principle: Prediction in Representation Space	2
1.3	The EMA Target Encoder	2
1.4	I-JEPA as the Image Example	3
1.5	Why JEPA Fits Gaze Modelling	3
2	Adaptation for Attentive Vision and Gaze	3
2.1	What Original SaccadeJEPA Provides	3
2.2	What Needs to Change for Gaze	4
2.3	The Cropper Substitution	4
2.4	How GazeCropper Works	5
2.5	Compatibility with SaccadeJEPA	5
2.6	Losses Reused from SaccadeJEPA	6
3	Possible Simulations and Results	6
3.1	Simulation Setup	7
3.2	Quantitative Results	7
3.3	Qualitative Scanpath Simulation	7
3.4	Interpretation	8
3.5	Limitations	9
4	Conclusion	9

1 General Description of JEPA

1.1 The Basic Idea

A Joint-Embedding Predictive Architecture (JEPA) is a self-supervised learning framework built around one central idea: instead of reconstructing missing data in pixel space, the model predicts missing information in a learned representation space. In plain terms, it does not try to reproduce every texture, colour fluctuation, or noise pattern in an image. It tries to predict the abstract state of what is missing.

This is different from pixel-reconstruction methods such as autoencoders, which must model all visible detail, including information that may be irrelevant for understanding the scene. It is also different from contrastive methods, which often depend on negative examples and hand-crafted augmentations. JEPA instead creates a predictive task directly from the input: observe one part, represent another part, and train the model to predict the second representation from the first.

1.2 Core Principle: Prediction in Representation Space

Let $\mathbf{x}$ be the observed part of an input and $\mathbf{y}$ the part to be predicted. A context encoder maps the observed part to a representation, while a target encoder maps the target part to another representation:

$$s_x = f_\theta(\mathbf{x}), \quad (1)$$

$$s_y = f_{\bar{\theta}}(\mathbf{y}), \quad (2)$$

The predictor then estimates the target representation from the context representation:

$$\hat{s}_y = g_\phi(s_x), \quad (3)$$

and training minimises the distance between the predicted and actual target representations:

$$\mathcal{L} = d(\hat{s}_y, s_y), \quad (4)$$

where $d(\cdot, \cdot)$ is usually a squared-error or Huber-style distance. The important point is that the loss is not computed between images. It is computed between embeddings. The model therefore learns to predict the abstract state of the target, not its exact pixels.

1.3 The EMA Target Encoder

A practical issue in joint-embedding methods is collapse: both encoders could learn to output nearly constant vectors, producing a low loss without useful representations. JEPA avoids this by making the two branches asymmetric. The context encoder is trained by gradient descent, but the target encoder is a slow-moving copy of it.

Concretely, the target encoder parameters $\bar{\theta}$ are updated by an exponential moving average (EMA) of the context encoder parameters θ :

$$\bar{\theta} \leftarrow \tau \bar{\theta} + (1 - \tau) \theta, \quad (5)$$

with τ close to one. In plain terms, the target encoder is not trained directly by back-propagation. It follows the context encoder slowly. This gives the predictor a stable target to chase while still allowing the target representation to improve over training.

1.4 I-JEPA as the Image Example

I-JEPA is the main image-based example of this idea. It divides an image into patch tokens, hides large contiguous target blocks, and gives the context encoder only the visible patches. The target encoder computes representations for the hidden blocks, and the predictor learns to infer those target representations from the visible context.

The important point for this report is not every architectural detail of I-JEPA, but the pattern it establishes:

`visible image region → predict representation of unseen image region.`

Because I-JEPA predicts representations rather than pixels, it encourages the encoder to learn semantic and spatially meaningful features. This makes it a useful conceptual bridge to gaze: a fixation also reveals only part of the visual scene, and the visual system must infer what lies outside the current high-acuity region.

1.5 Why JEPA Fits Gaze Modelling

The gaze interpretation follows naturally from the JEPA formulation. If $\mathbf{x}$ is the current fixation crop and $\mathbf{y}$ is the next fixation crop, then the task becomes:

`current fixation representation + movement information → next fixation representation.`

This is the bridge used in the rest of the report. Original SaccadeJEPA already applies JEPA to spatial shifts: it predicts the representation of a shifted crop from another crop and an encoded movement. The adaptation in this report keeps that prediction machinery, but changes where the shifts come from. Instead of one random perturbation, the model receives saliency-driven gaze transitions.

2 Adaptation for Attentive Vision and Gaze

2.1 What Original SaccadeJEPA Provides

The starting point is the `SaccadeJepa` model from the LumenPallidum JEPA repository [9]. It is an unpublished JEPA-style variant, separate from the repository’s standard `ViTJepa` implementation. Its pretext task is simple: given two spatially related crops from the same image, predict the target encoder’s representation of one crop from the context encoder’s representation of the other.

In plain words, original SaccadeJEPA does this:

`image -> one random crop pair -> encode -> predict target representation.`

The main components are already useful for gaze modelling:

- **Context encoder:** a ConvNeXt-Tiny network that encodes the first crop.

- **Target encoder:** an EMA copy of the context encoder that encodes the second crop without gradient updates.
- **Affine embedder:** a NeRF-style sinusoidal embedding of the spatial displacement between the two crops.
- **Predictor:** a three-layer MLP that receives the context representation plus the movement embedding and predicts the target representation.

The original cropper is the weak part for gaze. **SaccadeCropper** samples one random affine transformation per image. The sampled movement is not conditioned on image content, does not form a sequence, and does not remember where the model has already looked. It is useful as a representation-learning pretext task, but it is not yet an explicit gaze simulator.

2.2 What Needs to Change for Gaze

The biological interpretation of gaze requires three behaviours that original SaccadeJEPA does not provide:

- **Content-driven target selection.** The next crop should depend on the image, not on a uniform random draw.
- **Sequential fixations.** The model should produce a short scanpath, not only one independent crop pair.
- **Inhibition of return.** Recently visited locations should be temporarily suppressed so the model does not repeatedly sample the same point.

The conservative design choice is to change only the component responsible for choosing and producing crops. The encoder, target encoder, EMA update, predictor, affine embedder, and loss are kept from SaccadeJEPA. This keeps the adaptation easy to interpret: if the patched model learns, the learning signal comes from the same JEPA machinery, but applied to gaze-like transitions.

2.3 The Cropper Substitution

The adaptation is implemented in the project’s JEPA-reuse module. Its factory first builds a normal **SaccadeJepa** instance and then replaces one attribute:

```
sj.saccade_cropper = GazeCropper(...)
```

This is the central architectural change of the project. Before the swap, the model receives one random crop pair. After the swap, it receives a sequence of saliency-driven fixation transitions:

```
image -> saliency map -> fixation 1 -> fixation 2 -> ... -> JEPA
                                prediction.
```

GazeCropper preserves the same return signature as the upstream **SaccadeCropper**:

$$(\mathbf{view}_1, \mathbf{view}_2, \mathbf{affines}) = \text{cropper}(\mathbf{x}). \quad (6)$$

Because the rest of **SaccadeJepa.forward** sees the same three tensors, the upstream forward pass does not need to be rewritten.

2.4 How GazeCropper Works

For each image, **GazeCropper** performs a short explicit gaze loop:

1. Compute a saliency map from the image.
2. Initialise an inhibition-of-return mask to ones.
3. For T steps, sample the next fixation from saliency $\times$ IOR.
4. After each fixation, suppress that location with a Gaussian inhibition well and let older suppressions decay.
5. Convert the T fixations into $T - 1$ transitions ($\text{fix}_t \rightarrow \text{fix}_{t+1}$).

In the experiments below, $T = 5$, so each image produces four transitions. Each transition contributes a 128×128 crop at the current fixation, a 128×128 crop at the next fixation, and the normalised displacement between them. The predictor is therefore trained on:

`current fixation crop + movement embedding  $\rightarrow$  next fixation representation.`

The saliency source is pluggable: any module that maps an RGB image batch to a normalised saliency map can be used. This includes random saliency, centre bias, classical spectral-residual saliency, or Arash Khosropour’s trained ResNet saliency model. The ResNet model is the default for the final experiments; it is trained separately on FIND fixation maps and then frozen during JEPA training.

2.5 Compatibility with SaccadeJEPA

The substitution is safe because **GazeCropper** respects the two contracts expected by the upstream model.

First, the embedding dimension is unchanged. The upstream affine embedder expects a 36-dimensional NeRF-style movement embedding. This comes from translation order $L_t = 4$ and rotation order $L_r = 10$:

$$\text{affine_embed_dim} = L_t \cdot 4 + L_r \cdot 2 = 36. \quad (7)$$

GazeCropper reads these orders from the original cropper and produces the same embedding width. Rotation is set to zero, because gaze shifts translate the fixation window but do not rotate the world.

Second, the encoder inputs remain symmetric. Both the context encoder and the target encoder still receive 3-channel 128×128 RGB crops. This matters because the target encoder is an EMA copy of the context encoder. An earlier integrated-controller design used a 6-channel fovea-plus-periphery context branch with a 3-channel target branch; that would have made the EMA path asymmetric. The final design avoids that issue by keeping the upstream 3-channel crop interface.

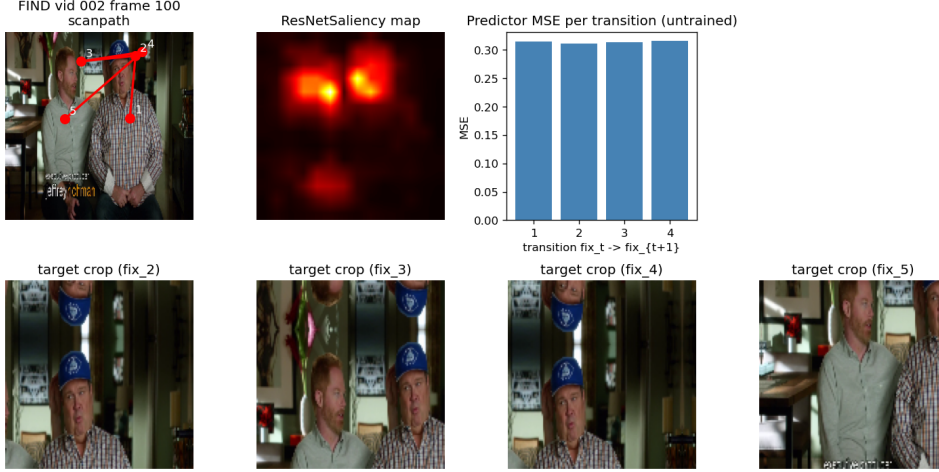


Figure 1: The patched **SaccadeJepa** on a FIND frame at 200×200 . **GazeCropper** consumes the frame and a saliency map, runs $T = 5$ saliency $\times$ IOR fixations, and emits four (**view**₁, **view**₂, **affine**) transitions in the same return format expected by upstream **SaccadeJEPA**. Every box after the cropper is reused from the original model.

2.6 Losses Reused from SaccadeJEPA

The training objective is also reused. The main term compares the predictor’s output with the EMA target encoder’s representation of the next fixation crop:

$$\mathcal{L}_{\text{pred}} = \text{HuberLoss}(\hat{s}_y, s_y). \quad (8)$$

The upstream cycle-consistency term asks the predictor to approximately undo a transition:

$$\mathcal{L}_{\text{cycle}} = \|\hat{s}_x - s_x\|_2^2. \quad (9)$$

VICReg variance and covariance terms are kept as collapse-prevention regularisers. In compact form, the total objective is:

$$\mathcal{L} = \mathcal{L}_{\text{pred}} + \lambda_{\text{cycle}} \mathcal{L}_{\text{cycle}} + \lambda_{\text{var}} \mathcal{L}_{\text{var}} + \lambda_{\text{cov}} \mathcal{L}_{\text{cov}}. \quad (10)$$

Thus the adaptation is deliberately minimal. The source of the two views changes from a random affine perturbation to a saliency-driven gaze transition; the JEPA prediction machinery remains the same.

3 Possible Simulations and Results

The feasible simulation is a self-supervised gaze-transition prediction task on FIND [7]. The model receives a static video frame, uses **GazeCropper** to generate a short saliency-driven scanpath, and trains **SaccadeJEPA**’s predictor to anticipate the representation of the next fixation crop. This does not yet claim to reproduce human scanpaths exactly. It tests a more basic question: after the cropper substitution, can the original JEPA machinery still learn?

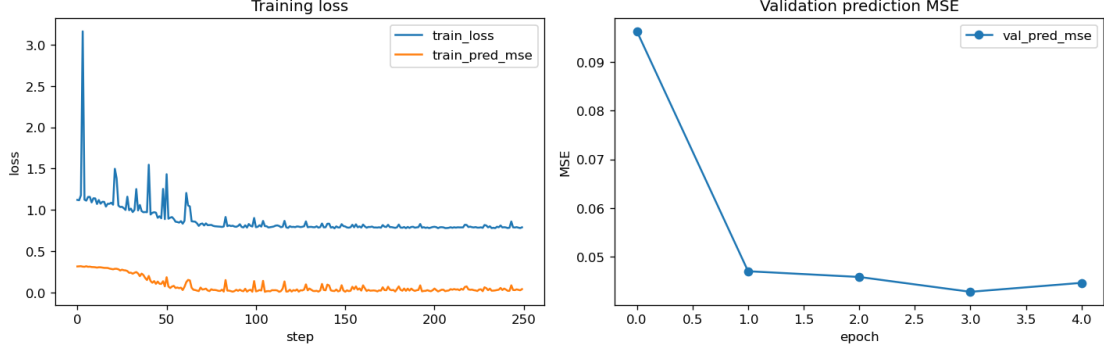


Figure 2: Training loss and prediction MSE for the patched **SaccadeJepa** on FIND. Five epochs, batch size 4, four frames per training video, on CPU. The total loss combines a Huber prediction term, a cycle term, and the VICReg variance/covariance regulariser. The prediction MSE is read directly from the predictor against the EMA target encoder. Per-batch values are shown lightly; the heavy line is a running mean.

3.1 Simulation Setup

Frames are decoded from FIND raw MP4 videos and resized to 200×200 . The shared split is 50 training videos, 6 validation videos, and 6 test videos. For the demonstration run, each training video contributes four sampled frames per epoch. With batch size 4 and $T = 5$ fixations per frame, the predictor sees 16 fixation transitions per batch.

Training uses the same loss family described in Sec. 2.6: Huber prediction loss [4], cycle consistency, and VICReg variance/covariance regularisation [1]. AdamW is used with learning rate $5 \cdot 10^{-5}$ and EMA decay $\tau = 0.996$ [2]. The only practical adjustment is that the VICReg weights are reduced for the small CPU batch size: $\lambda_{\text{var}} = 1.0$, $\lambda_{\text{cov}} = 0.1$, and $\gamma = 1.0$. The upstream defaults were tuned for a much larger effective batch.

The run lasts five CPU epochs. Figure 2 shows that both total loss and prediction MSE decrease over training. The key curve is prediction MSE, because it measures how close the predictor is to the EMA target representation of the next fixation.

3.2 Quantitative Results

The main result is the validation prediction MSE. Before training, the patched model gives a validation MSE of 0.2996. After five epochs, the matched validation pass gives 0.0462. This is an 84.6% reduction. The result means that the predictor learned to anticipate the next-fixation representation under the saliency-driven cropper.

3.3 Qualitative Scanpath Simulation

Figure 3 shows four scanpaths sampled on held-out FIND test frames. The cropper itself is not trained by the JEPA loss; its fixations come from the frozen saliency source and the IOR mask. The figure is therefore not evidence that the model has learned a human scanpath policy. Its role is more limited: it shows that the substituted cropper produces structured, sequential fixations on real video frames, and that the crops passed into SaccadeJEPA are visually meaningful.

Phase	Prediction MSE
Untrained, validation pool ($n_{\text{val}}=24$)	0.2996
Validation, end of epoch 1	0.0963
Validation, end of epoch 2	0.0471
Validation, end of epoch 3	0.0459
Validation, end of epoch 4	0.0428
Validation, end of epoch 5	0.0447
Trained, validation pool (matched val pass on <code>final.pt</code>)	0.0462

Table 1: Predictor mean-squared error against the EMA target encoder, before and after a five-epoch training run on FIND. All numbers are averaged across the six held-out validation videos sampled at four frames per video (batch size 4, $T=5$ fixations per frame), using Arash’s trained **ResNetSaliency** as the saliency source. The last row re-loads the final checkpoint and replays the validation pass in matched conditions: the training-time number recorded at end of epoch 5 (0.0447) and the post-hoc reproduction (0.0462) agree to within batch-level noise.



Figure 3: Fixation sequences produced by the trained **GazeCropper** on four FIND test frames. Numbered red markers are the five fixations in temporal order; lines connect consecutive fixations. The saliency source is Arash’s trained **ResNetSaliency** (ResNet-18 backbone with a 1×1 -convolution head, drawn from the shared **gazejepa.saliency** package), and the inhibition mask uses the same $\sigma_{\text{IOR}}=20$ px and decay 0.7 as in the training run.

3.4 Interpretation

These results answer the feasibility question positively. With a one-component substitution, SaccadeJEPA can be made to operate on explicit gaze transitions, and the original prediction machinery still converges. The decrease in MSE is the same kind of JEPA signal as in the original model: the predictor approaches the EMA target representation. The difference is that the two views now come from consecutive saliency-driven fixations rather than one random affine crop.

The result should be interpreted carefully. The model is not trained with a human scanpath loss, and it does not receive gradients through the discrete fixation choices. It is therefore better described as a reusable JEPA backbone for gaze-transition prediction, not as a complete learned gaze policy.

3.5 Limitations

The main limitations are:

- **Static frames.** FIND is a video dataset, but this simulation runs $T = 5$ fixations on a single frozen frame.
- **No fixation duration.** The model predicts where the next crop is, not how long the eye stays there.
- **Frozen saliency.** Arash’s ResNetSaliency is used as a fixed prior. It is not updated by the JEPA loss.
- **Small training budget.** The run is five epochs on CPU, so it is a feasibility check rather than a tuned final model.
- **No learned policy.** A future version could learn the fixation policy directly, for example with a recurrent attention model in the spirit of Mnih et al. [10].

4 Conclusion

This report asked whether SaccadeJEPA can be reused in a less obscure way for explicit gaze simulation. The answer is yes, within a carefully limited scope. Original SaccadeJEPA already contains the useful JEPA machinery: a context encoder, an EMA target encoder, a movement embedding, a predictor, and a representation-level prediction loss. Its limitation is the source of the views: one random crop pair per image.

The proposed adaptation changes that source of views. Replacing **SaccadeCropper** with **GazeCropper** turns the model from:

image -> random crop pair -> predict representation

into:

image -> saliency-driven fixation sequence -> predict next fixation.

The experiment on FIND shows that this substitution is compatible with the upstream loss: validation prediction MSE drops from 0.2996 to 0.0462 after five CPU epochs. This is not a claim that the model fully matches human gaze, but it is strong feasibility evidence that SaccadeJEPA can be reused as a representation-space predictor over explicit gaze transitions.

References

- [1] A. Bardes, J. Ponce, and Y. LeCun. VICReg: Variance-Invariance-Covariance regularization for self-supervised learning. In *International Conference on Learning Representations (ICLR)*, 2022.
- [2] J.-B. Grill, F. Strub, F. Altché, C. Tallec, P. H. Richemond, E. Buchatskaya, C. Doersch, B. Avila Pires, Z. D. Guo, M. Gheshlaghi Azar, B. Piot, K. Kavukcuoglu, R. Munos, and M. Valko. Bootstrap your own latent: a new approach to self-supervised learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

- [3] X. Hou and L. Zhang. Saliency detection: a spectral residual approach. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2007.
- [4] P. J. Huber. Robust estimation of a location parameter. *Annals of Mathematical Statistics*, 35(1):73–101, 1964.
- [5] L. Itti, C. Koch, and E. Niebur. A model of saliency-based visual attention for rapid scene analysis. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 20(11):1254–1259, 1998.
- [6] R. M. Klein. Inhibition of return. *Trends in Cognitive Sciences*, 4(4):138–147, 2000.
- [7] Y. Liu and M. Xu. FIND: a database for multi-face tracking with fixation data on multiparty conversation videos. In *Proceedings of the IEEE International Conference on Multimedia and Expo (ICME)*, 2016.
- [8] Z. Liu, H. Mao, C.-Y. Wu, C. Feichtenhofer, T. Darrell, and S. Xie. A ConvNet for the 2020s. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022.
- [9] LumenPallidium. JEPA: a PyTorch implementation of joint-embedding predictive architectures, including SaccadeJEPA. Software repository, 2024. <https://github.com/LumenPallidium/jepa>, accessed 2026.
- [10] V. Mnih, N. Heess, A. Graves, and K. Kavukcuoglu. Recurrent models of visual attention. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2014.
- [11] M. I. Posner and Y. Cohen. Components of visual orienting. In H. Bouma and D. G. Bouwhuis, editors, *Attention and Performance X: Control of Language Processes*, pages 531–556. Erlbaum, 1984.